

# Lecture 2: Term Vocabulary & Posting Lists

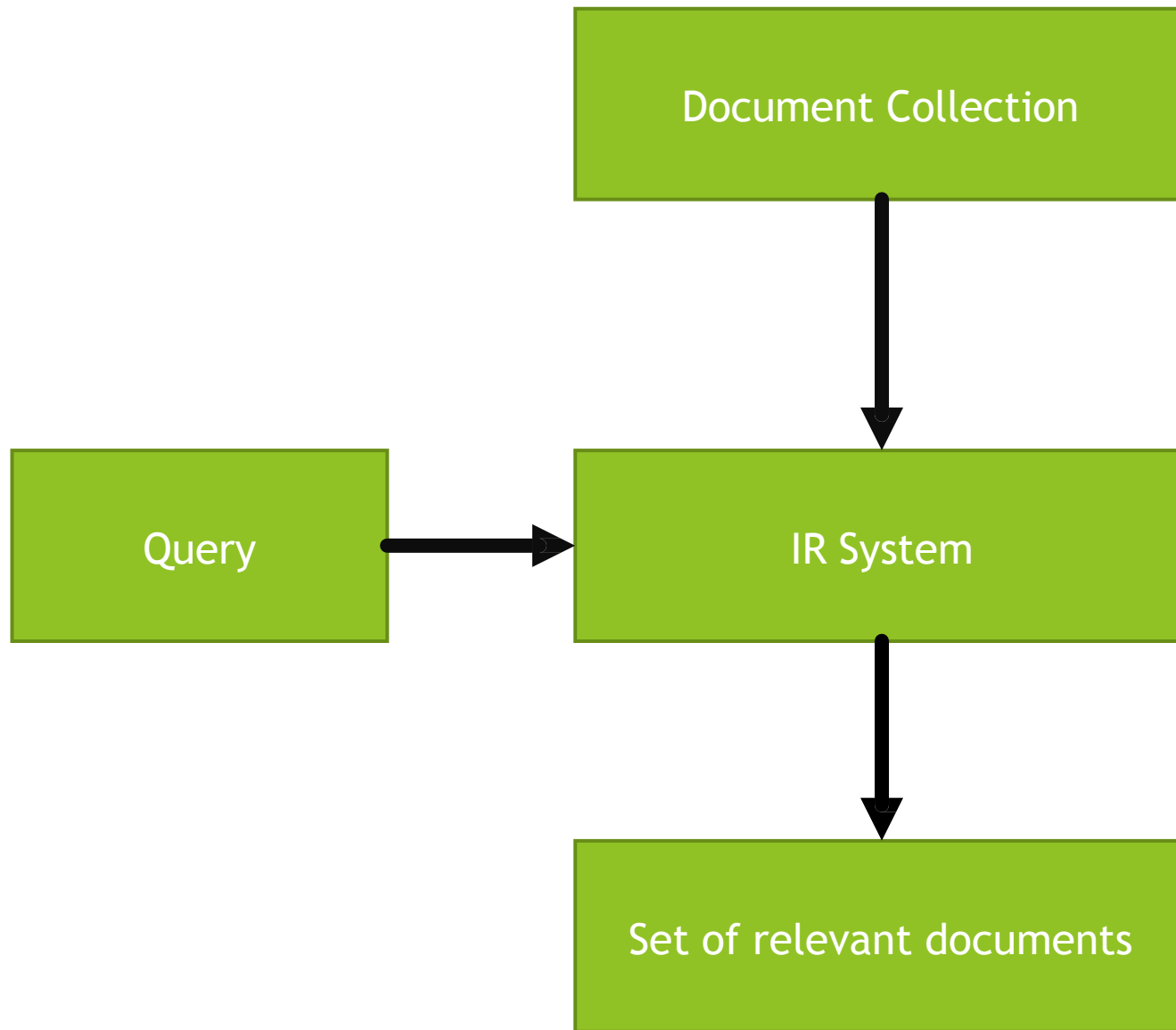
## Information Retrieval and Web Analytics

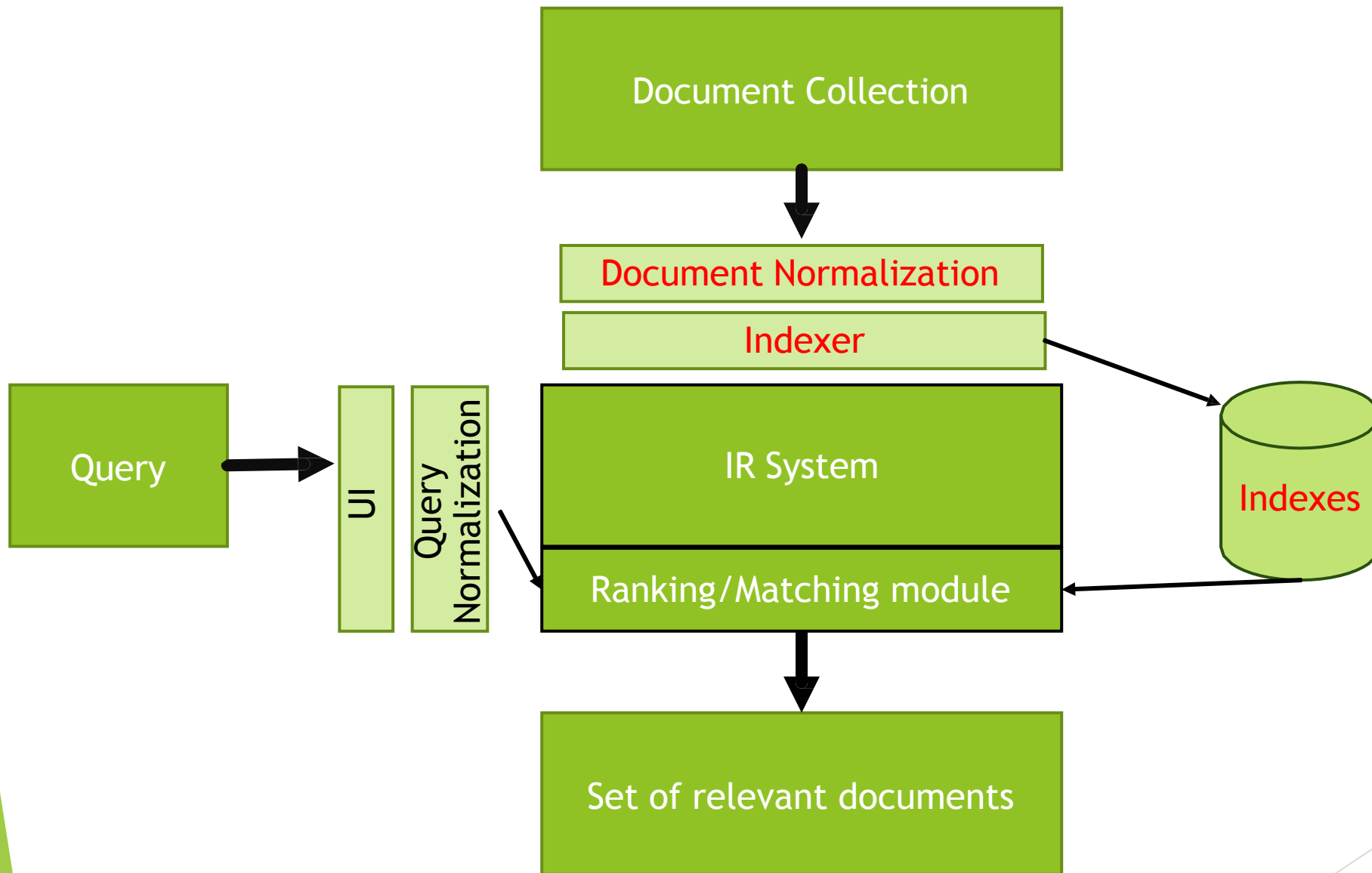
### Recap

- Introduction to IR
- Incidence Matrix
- Inverse Index
  - Sorting
  - Term Frequency
- Query Optimization

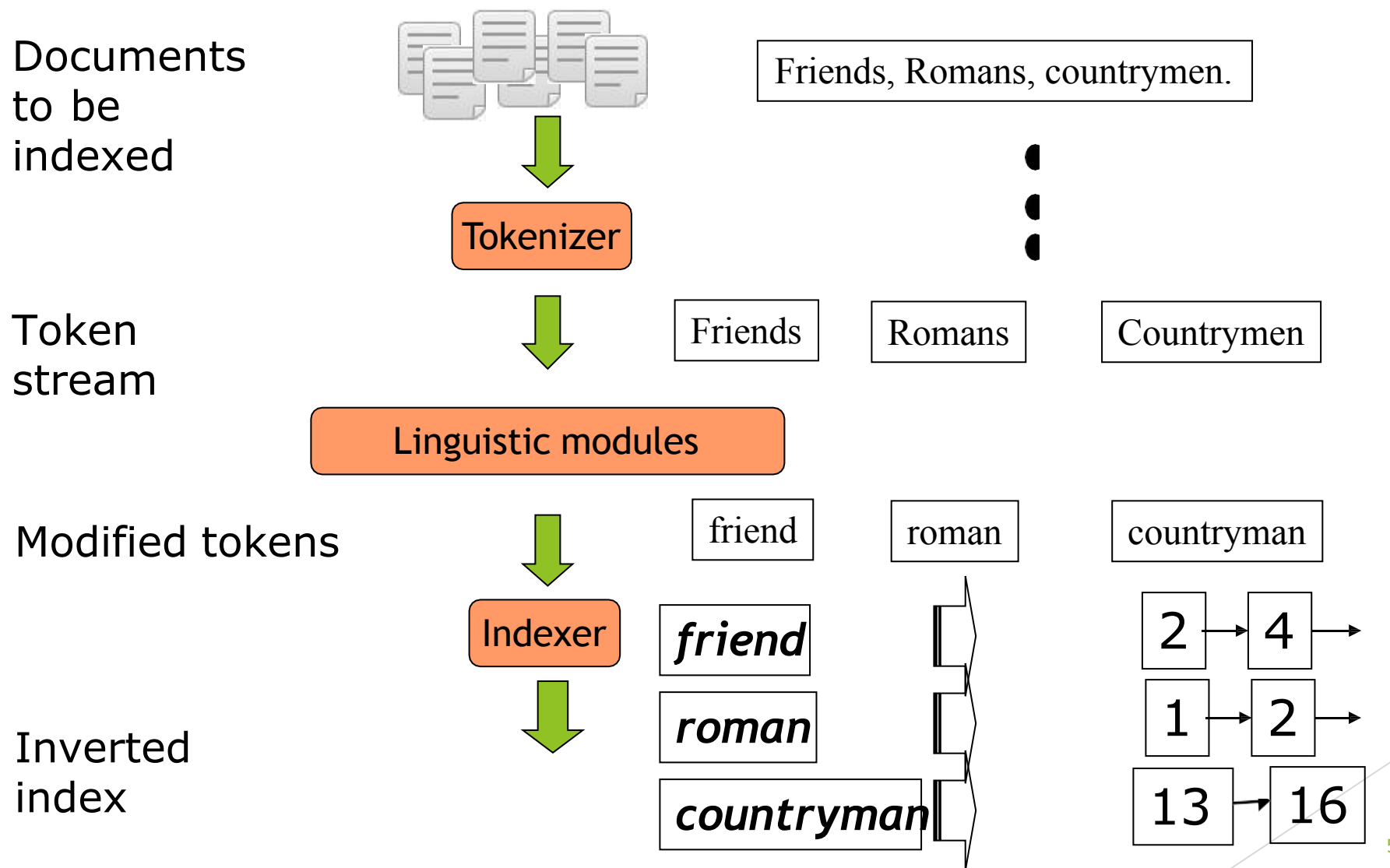
# Today's Lecture

- Preprocessing to form the term vocabulary
  - Documents
  - Tokenization
  - Stop Words
  - Normalization
  - Lemmatization
  - Stemming
- Postings
  - Faster merges: skip lists
  - Positional postings and phrase queries





# Recall the basic indexing pipeline



# Parsing a document

- ▶ A document is stored as a sequence of bytes
- ▶ Need to convert byte sequence into a linear sequence of characters
- ▶ What character set is in use?
  - ▶ (CP1252, UTF-8, ...)
- ▶ What format is it in?
  - ▶ pdf/word/excel/html/XML?
- ▶ What language is it in?
  - ▶ Arabic – Mixed order characteristics but not linear sequence of characters

Each of these is a classification problem, which we will study later in the course.

But these tasks are often done heuristically ...

# Complications: Format/language

- ▶ Characters should be decoded out of some file format
- ▶ Documents being indexed can include docs from many different languages
  - ▶ A single index may contain terms from many languages.
- ▶ Sometimes a document or its components can contain multiple languages/formats
  - ▶ French email with a German pdf attachment.
  - ▶ French email quote clauses from an English-language contract
- ▶ There are commercial and open source libraries that can handle a lot of this stuff.
- ▶ However have to deal with them case by case

# Complications: What is a document?

We return from our query “documents” but there are often interesting questions of grain size:

What is a unit document?

- ▶ A file?
- ▶ An email? (Perhaps one of many in a single mbox file)
  - ▶ What about an email with 5 attachments?
- ▶ A group of files (e.g., PPT or LaTeX split over HTML pages)
- ▶ A book or a chapter or even further reduction??



# Tokenization

- **Tokenization** is the act of breaking up a sequence of strings into pieces such as words, keywords, phrases, symbols and other elements called tokens.
- **Token** - A token is an instance of a sequence of characters in some particular document that are grouped together as a use full semantic unit for processing.
- **Input**: “*Friends, Romans and Countrymen*”
- **Output**: Tokens -
  - *Friends*
  - *Romans*
  - *and*
  - *Countrymen*
- **Type** - A type is the class of all TERM tokens containing the same character sequence.
  - increase in home sales in july - 6 tokens and 5 types
- Each such token is now a candidate for an index entry, after further processing
- But what are valid tokens to emit? May be “in”
- **Term** - **type that is included in the IR system’s dictionary** - 4 terms if “in” omitted.
- Eg - eating and drinking and dancing

# Challenges in Tokenization

- What are the correct tokens to use?
  - E.g. Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing
- How to tokenize O'Neill?
  - neill /oneill/o'neill/o' neill/o neill
- How to tokenize aren't?
  - aren't/aren't/are n't/aren t
- Simplest strategy
  - split on all non-alphanumeric characters
  - Consider query neil AND capital
  - Consider query o'neill AND capital
- Rule of thumb:
  - use the same tokenizer for both documents and text(query)

# Challenges in Tokenization

- Some domain-specific challenges
  - Programming languages - C++ and C#
  - Aircraft names - B-52
  - Tv shows - M\*A\*S\*H
  - Email addresses
  - Urls
  - IP addresses
  - Package tracking numbers
- Just omitting these from indexing seems to be the easiest option??
- How to handle hyphenation?
  - co-education
  - Hewlett-Packard
  - the hold-him-back-and-drag-him-away maneuver
- Not possible to have generic rules
- Have to use some heuristics
  - Allow short hyphenated prefixes only

# Challenges in Tokenization

- Even splitting on whitespace is non-trivial
  - Names: Los Angeles
  - Borrowed foreign phrases: bon appetit
  - Compounds written with or without space: white space vs whitespace
  - Phone numbers
  - Dates: 3/20/91 *Mar. 12, 1991* 20/3/91
- Consider lower case, lower-case, lowercase
  - Can encourage users to use hyphenation whenever possible?
  - System checks for all three forms
  - But this requires user training

# Challenges in Tokenization

- ▶ German writes compound nouns without spaces
  - ▶ *Lebensversicherungsgesellschaftsangestellter* - 'life insurance company employee'
- ▶ Chinese and Japanese have no spaces between words:
  - ▶ 莎拉波娃现在居住在美国东南部的佛罗里达。
  - ▶ Not always guaranteed a unique tokenization
  - ▶ Need to perform word segmentation
    - ▶ Use a lexicon or supervised machine-learning
- ▶ Further complicated in Japanese, with multiple alphabets intermingled
  - ▶ Dates/amounts in multiple formats

# Challenges in Tokenization

- ▶ Arabic (or Hebrew) is basically written right to left, but with certain items like numbers written left to right
- ▶ Words are separated, but letter forms within a word form complex ligatures
- ▶ استقلت الجزائر في سنة 1962 بعد 132 عاما من الاحتلال الفرنسي.
- ▶                                      ← →   ← →                                      ← start
- ▶ ‘Algeria achieved its independence in 1962 after 132 years of French occupation.’
- ▶ With Unicode, the surface presentation is complex, but the stored form is straightforward
- ▶ Our recent experience with Tamil
  - ▶ – Some two unicode letters in tamil are splitted using tokenizer.
  - ▶ – Some initials of the names are splitted as words.
- ▶ E.g. -ஈ. எம். எம். கசீர் ( It should be H.M.M. Kaseer)
- ▶ some decimals s.a. 100.4 are tokenized as “100” “.” “4”
- ▶ Some date patterns are tokenized

# Stop words

- ▶ **Stop words:** Extremely common words which would appear to be of little value in helping select documents matching a user need.
- ▶ With a stop list, you exclude from the dictionary entirely the *commonest* words. Intuition:
  - ▶ They have little semantic content: *the, a, and, to, be*
  - ▶ There are a lot of them: ~30% of postings for top 30 words
- ▶ The general strategy for determining a stop list is to sort the terms by *collection frequency* (the total number of times each term appears in the document collection), and then to take the most frequent terms, often hand-filtered for their semantic content relative to the domain of the documents being indexed, as a *stop list*
- ▶ But the trend is away from doing this:
  - ▶ You need them for:
    - ▶ Phrase queries: “King of Denmark”
    - ▶ Various song titles, etc.: “Let it be”, “To be or not to be”
    - ▶ “Relational” queries: “flights to London”
- ▶ Length of practically used stoplists has shrunk over the years.
- ▶ Most web search engines do index stop words.

# Normalization to terms

- ▶ ***Token normalization*** is the process of canonicalizing tokens so that matches occur despite superficial differences in the character sequences of the tokens.
- ▶ We may need to “normalize” words in indexed text as well as query words into the same form
  - ▶ We want to match ***U.S.A.*** and ***USA***
- ▶ Result is terms: a **term** is a (normalized) word type, which is an entry in our IR system dictionary
- ▶ We most commonly implicitly define equivalence classes of terms by, e.g.,
  - ▶ deleting periods to form a term
    - ▶ ***U.S.A., USA*** ☾ ***USA***
  - ▶ deleting hyphens to form a term
    - ▶ ***anti-discriminatory, antidiscriminatory*** ☾ ***antidiscriminatory***



# Normalization to terms

- Alternatively, can maintain relations between un-normalized tokens
  1. Index un-normalized tokens and to maintain a query expansion list of multiple vocabulary entries to consider for a certain query term.
    - A query term is then effectively a disjunction of several postings lists
  2. Can also perform the expansion during index construction
    - E.g. When the document contains automobile, we index it under car as well
- Issues in Normalizing
  1. needs more processing
  2. needs more space
  3. How much normalizing?

# Different Forms of Normalization

- Accents and diacritics
  - Need to match cliché and cliché (might not be true for some cases)
- Capitalization/case-folding
  - Simplest strategy: make everything lowercase
    - But this cannot distinguish proper nouns
    - Users often type queries in lowercase
  - Alternative: only make some letters lowercase
  - Or use a machine learning technique to learn the true case (true casting)
- Other issues in English
  - Shortened words (ne'er vs never)
  - British English vs American English
  - Different date/time formats

# Normalization: other languages

- ▶ Accents: e.g., French *résumé* vs. *resume*.
- ▶ Umlauts: e.g., German: *Tuebingen* vs. *Tübingen*
  - ▶ Should be equivalent
- ▶ Most important criterion:
  - ▶ How are your users like to write their queries for these words?
- ▶ Even in languages that standardly have accents, users often may not type them
  - ▶ Often best to normalize to a de-accented term
    - ▶ *Tuebingen, Tübingen, Tubingen* ☾ *Tubingen*

# Normalization: other languages

- ▶ Normalization of things like date forms
  - ▶ 7月30日 vs. 7/30
  - ▶ *Japanese vs. Western*
- ▶ To normalize these, you might convert them both to a common format like 2024-07-30 (ISO format).
- ▶ Tokenization and normalization may depend on the language and so is intertwined with language detection

***Morgen will ich in MIT...***

Is this German “mit”?
- ▶ Crucial: Need to “normalize” indexed text as well as query terms **identically**.

# Case folding

- ▶ Reduce all letters to lower case
  - ▶ exception: upper case in mid-sentence?
    - ▶ e.g., General Motors
    - ▶ Fed vs. fed
    - ▶ SAIL vs. sail
  - ▶ Often best to lower case everything, since users will use lowercase regardless of ‘correct’ capitalization...
- ▶ Longstanding Google example:
  - ▶ Query C.A.T.
  - ▶ #1 result is for “cats” (well, Lolcats) not Caterpillar Inc.

# Normalization to terms

- ▶ An alternative to equivalence classing is to do asymmetric expansion
- ▶ An example of where this may be useful
  - ▶ Enter: ***window***      Search: ***window, windows***
  - ▶ Enter: ***windows***      Search: ***Windows, windows, window***
  - ▶ Enter: ***Windows***      Search: ***Windows***
- ▶ Potentially more powerful, but less efficient

# Thesauri and soundex

- ▶ Do we handle synonyms and homonyms?
  - ▶ E.g., by hand-constructed equivalence classes
    - ▶ *car* = *automobile*    *color* = *colour*
  - ▶ We can rewrite to form equivalence-class terms
    - ▶ When the document contains *automobile*, index it under *car-automobile* (and vice-versa)
  - ▶ Or we can expand a query
    - ▶ When the query contains *automobile*, look under *car* as well (take union of two postings lists of terms automobile & car )
- ▶ What about spelling mistakes?
  - ▶ One approach is Soundex, which forms equivalence classes of words based on phonetic heuristics

# Stemming

- ▶ Reduce terms to their “roots” before indexing
- ▶ “Stemming” is a crude heuristic process that chops off the ends of words in the hope of achieving what “principled” lemmatisation attempts to do with a lot of linguistic knowledge.
  - ▶ language dependent, but fast and space-efficient
- ▶ e.g., ***automate(s), automatic, automation*** all reduced to ***automat***.

***for example compressed and compression are both accepted as equivalent to compress.***



for example compress and  
compress are both accepted  
as equivalent to compress



# Porter's algorithm

- ▶ Commonest algorithm for stemming English
  - ▶ Results suggest it's at least as good as other stemming options
- ▶ Conventions + 5 phases of reductions
  - ▶ phases applied sequentially
  - ▶ each phase consists of a set of commands
  - ▶ sample convention: *Of the rules in a compound command, select the one that applies to the longest suffix.*

# Typical rules in Porter

- ▶ *sses* → *ss* – Ex: *caresses* → *caress*
- ▶ *ies* → *i* – Ex: *ponies* → *poni*
- ▶ *ational* → *ate* – Ex: *International* → *internate*
- ▶ *tional* → *tion* – conditional    *condition*

# Other stemmers

- ▶ Other stemmers exist:
  - ▶ Lovins stemmer
    - ▶ <http://www.comp.lancs.ac.uk/computing/research/stemming/general/lovins.htm>
    - ▶ Single-pass, longest suffix removal (about 250 rules)
  - ▶ Paice/Husk stemmer
  - ▶ Snowball
- ▶ Full morphological analysis (lemmatization)
  - ▶ At most modest benefits for retrieval
- ▶ Do stemming and other normalizations help?
  - ▶ English: very mixed results. **Helps recall but harms precision**
    - ▶ operative (dentistry) ⇒ oper
    - ▶ operational (research) ⇒ oper
    - ▶ operating (systems) ⇒ oper
  - ▶ Definitely useful for Spanish, German, Finnish, ...
  - ▶ 30% performance gains for Finnish!

# Lemmatization

- ▶ Inflectional morphology (cutting ☾ cut)
- ▶ Derivational morphology (destruction ☾ destroy)
- ▶ *Lemmatization* usually refers to doing things properly with the use of a vocabulary and morphological analysis of words, normally aiming to **remove inflectional endings** only and to return the **base or dictionary form of a word**, which is known as the *lemma*.
- ▶ Reduce inflectional/variant forms to base form
- ▶ E.g.,
  - ▶ *am, are, is* ☾ *be*
  - ▶ *car, cars, car's, cars'* ☾ *car*
- ▶ *the boy's cars are different colors* ☾ *the boy car be different color*
- ▶ Lemmatization implies doing “proper” reduction to dictionary headword form (the **lemma**)

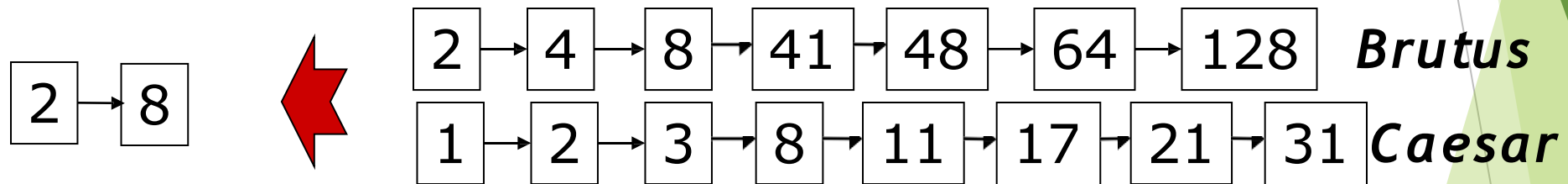
# Language-specificity

- ▶ The above methods embody transformations that are
  - ▶ Language-specific, and often
  - ▶ Application-specific
- ▶ These are “plug-in” add-ons to the indexing process
- ▶ Both open source and commercial plug-ins are available for handling these

# FASTER POSTINGS MERGES: SKIP POINTERS/SKIP LISTS

# Recall basic merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries

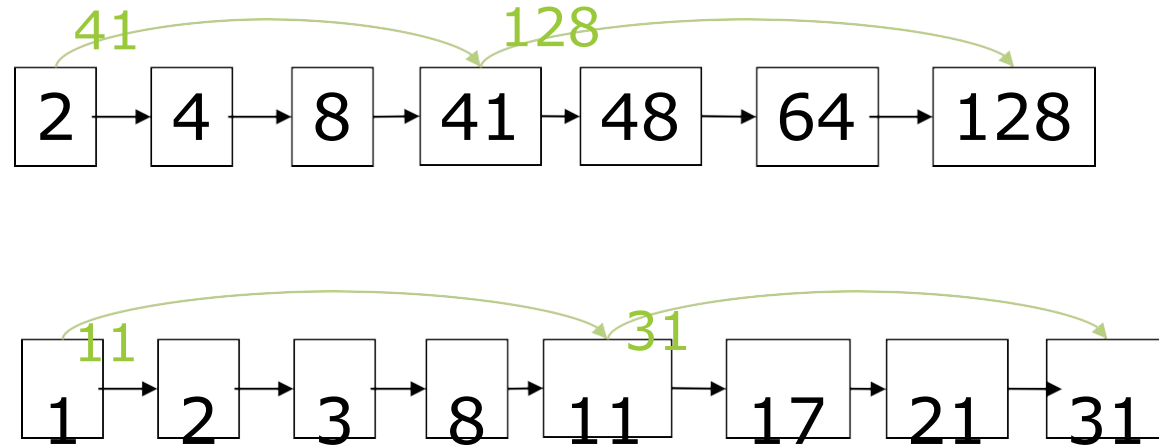


If the list lengths are  $m$  and  $n$ , the merge takes  $O(m+n)$  operations.

Can we do better?

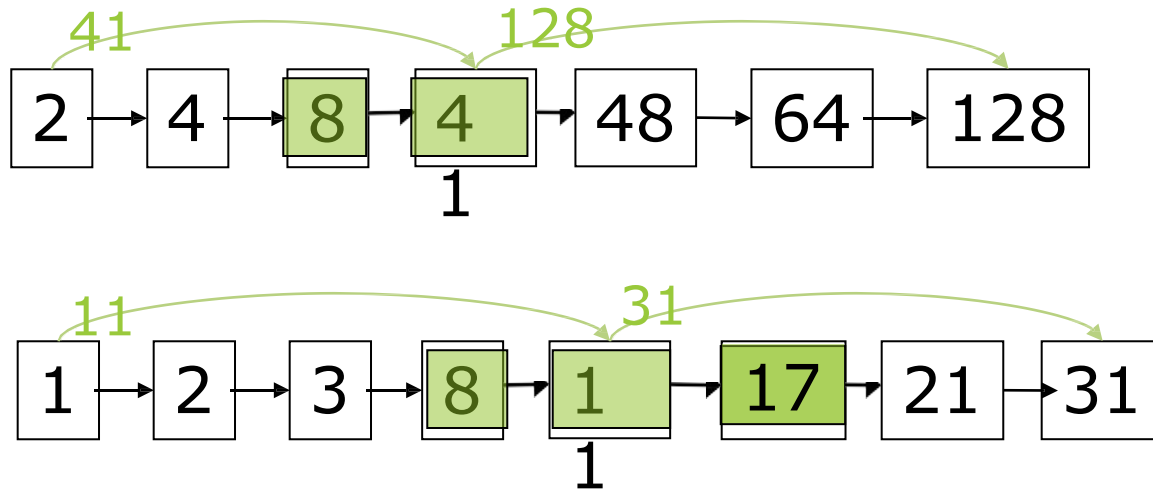
Yes (if the index isn't changing too fast).

## Augment postings with skip pointers (at indexing time)





# Query processing with skip pointers



```

INTERSECTWITHSKIPS( $p_1, p_2$ )
1   $answer \leftarrow \{ \}$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $docID(p_1) = docID(p_2)$ 
4      then  $ADD(answer, docID(p_1))$ 
5           $p_1 \leftarrow next(p_1)$ 
6           $p_2 \leftarrow next(p_2)$ 
7  else if  $docID(p_1) < docID(p_2)$ 
8      then if  $hasSkip(p_1)$  and  $(docID(skip(p_1)) \leq docID(p_2))$ 
9          then while  $hasSkip(p_1)$  and  $(docID(skip(p_1)) \leq docID(p_2))$ 
10             do  $p_1 \leftarrow skip(p_1)$ 
11          else  $p_1 \leftarrow next(p_1)$ 
12  else if  $hasSkip(p_2)$  and  $(docID(skip(p_2)) \leq docID(p_1))$ 
13      then while  $hasSkip(p_2)$  and  $(docID(skip(p_2)) \leq docID(p_1))$ 
14          do  $p_2 \leftarrow skip(p_2)$ 
15      else  $p_2 \leftarrow next(p_2)$ 
16  return  $answer$ 
    
```

Figure 2.10: Postings lists intersection with skip pointers.

Suppose we've stepped through the lists until we process **8** on each list. We match it and advance.

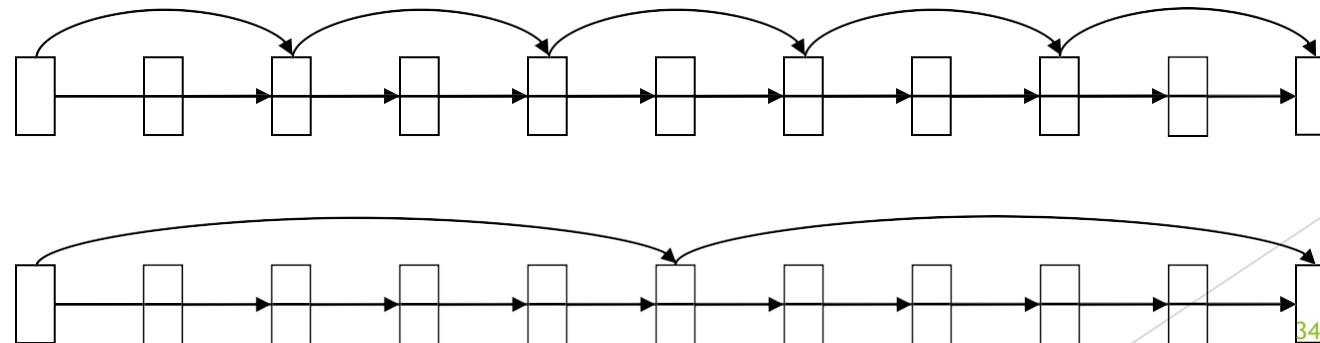
We then have **41** and **11** on the lower. **11** is smaller.

But the skip successor of **11** on the lower list is **31**,  
~~so~~ we can skip ahead past the intervening  
 postings.

# Where do we place skips?

## ► Tradeoff:

- More skips 🤔 shorter skip spans 🦋 more likely to skip. But lots of comparisons to skip pointers.
- Fewer skips 🤔 few pointer comparison, but then long skip spans 🦋 can not use it very often
- Skip pointers used to help a lot, but with today's fast CPUs, they don't help that much anymore.



# Placing skips

- ▶ Simple heuristic: for postings of length  $L$ , use  $\lceil L \rceil$  evenly-spaced skip pointers [Moffat and Zobel 1996]
- ▶ This ignores the distribution of query terms.
- ▶ Easy if the index is relatively static; harder if  $L$  keeps changing because of updates.
- ▶ This definitely used to help; with modern hardware it may not unless you're memory-based [Bahle et al. 2002]
  - ▶ The I/O cost of loading a bigger postings list can outweigh the gains from quicker in memory merging!

# Phrase queries

- ▶ We want to answer a query such as [cambridge university] – as a phrase.
- ▶ “**The Duke of Cambridge recently went for a term-long course to a famous university**” should not be a match.
- ▶ About 10% of web queries are phrase queries.
- ▶ Consequence for inverted indexes: no longer sufficient to store docIDs in postings lists.
- ▶ Two ways of extending the inverted index:
  - ▶ Biword index
  - ▶ Positional index

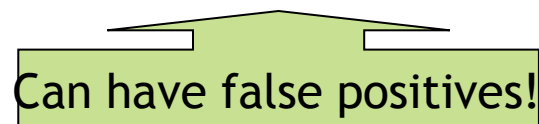
# Solution 1: Biword indexes

- ▶ Index every consecutive pair of terms in the text as a phrase
- ▶ For example the text “Friends, Romans, Countrymen” would generate the biwords
  - ▶ *friends romans*
  - ▶ *romans countrymen*
- ▶ Each of these biwords is now a dictionary term
- ▶ Two-word phrase query-processing is now immediate.

# Longer phrase queries

- ▶ A long phrase like “**cambridge university west campus**” can be represented as the Boolean query.
- ▶ **cambridge university AND university west AND west campus**
- ▶ We need to do post-filtering of hits to identify subset that actually contains the 4-word phrase.

Without the position of the term in specific doc, we cannot verify that the docs matching the above Boolean query do contain the phrase.



Can have false positives!

# Issues for biword indexes

- ▶ False positives, as noted before
- ▶ Index blowup due to bigger dictionary
  - ▶ Infeasible for more than biwords, big even for them

## Solution 2: Positional indexes

- ▶ Positional indexes are a more efficient alternative to biword indexes.
- ▶ Postings lists in a nonpositional index: each posting is just a docID
- ▶ Postings lists in a positional index: each posting is a docID and a list of positions (offsets)



## Solution 2: Positional indexes

- In the postings, store, for each ***term*** the position(s) in which tokens of it appear:

<***term***, number of docs containing ***term***;

*doc1*: position1, position2 ... ;

*doc2*: position1, position2 ... ;

etc.>

# Positional index example

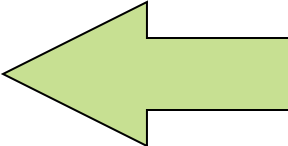
<*be*: 993427;

*1*: 7, 18, 33, 72, 86, 231;

*2*: 3, 149;

*4*: 17, 191, 291, 430, 434;

*5*: 363, 367, ...>



Which of docs *1,2,4,5*  
could contain "*to be*  
*or not to be*"?

# Processing a phrase query

- ▶ Extract inverted index entries for each distinct term: **to**, **be**.
- ▶ Merge their *doc:position* lists to enumerate all positions with “**to be**”.
  - ▶ **to**:
    - ▶ 2:1,17,74,222,551; 4:8,16,190,429,433;  
7:13,23,191; ...
  - ▶ **be**:
    - ▶ 1:17,19; 4:17,191,291,430,434; 5:14,19,101; ...
- ▶ Same general method for proximity searches

# Proximity queries

- ▶ employment /k place ; k=4
- ▶ Again, here, /k means “within  $k$  words of”.
- ▶ Find all documents that contain employment and place within 4 words of each other.
- ▶ HIT: **Employment** agencies that **place** healthcare workers are seeing growth.
- ▶ NO HIT: **Employment** agencies that have learned to adapt now **place** healthcare workers.

# Positional index size

- ▶ A positional index expands postings storage *substantially*
  - ▶ Even though indices can be compressed
- ▶ Nevertheless, a positional index is now standardly used because of the power and usefulness of phrase and proximity queries.
- ▶ The complexity will be  $O(T)$  but not  $O(N)$ .
  - ▶  $N$  – Number of documents;  $T$  – Number of Terms

# Positional index size

- ▶ Need an entry for each occurrence of the term, not just once per document
- ▶ Index size depends on average document size (because when doc size increase eventually no of terms in doc also increase)
  - ▶ Average web page has <1000 terms
  - ▶ SEC filings, books, even some epic poems ... easily 100,000 terms
- ▶ Consider a term with frequency 0.1% (t appears once every 1000 terms)

Why?

| Document size | Postings | Positional postings |
|---------------|----------|---------------------|
| 1000          | 1        | 1                   |
| 100,000       | 1        | 100                 |

# Rules of thumb

- ▶ A positional index is 2–4 as large as a non-positional index
- ▶ Positional index size 35–50% of volume of original text
  - ▶ Caveat: all of this holds for “English-like” languages

# Combination schemes

- ▶ Biword indexes and positional indexes can be profitably combined.
- ▶ Many biwords are extremely frequent: **Michael Jackson, Britney Spears** etc
- ▶ For these biwords, increased speed compared to positional postings intersection is substantial.
- ▶ Combination scheme: Include frequent biwords as vocabulary terms in the index. Do all other phrases by positional intersection.
- ▶ Williams et al. (2004) evaluate a more sophisticated mixed indexing scheme. Faster than a positional index, at a cost of 26% more space for index.